

CI/CD fundamentals

COMPLETE LEARNING GUIDE

What You'll Learn

Continuous Integration and Continuous Deployment automate the steps between 'a developer pushes code' and 'that code is running safely in production,' replacing manual, error-prone, and inconsistently-applied processes with a repeatable pipeline. This guide covers what CI/CD actually automates and why that matters, the typical anatomy of a pipeline (build, test, deploy stages), writing a GitHub Actions workflow concretely, using automated tests and linting as real quality gates rather than a formality, caching dependencies to keep pipelines fast, the major deployment strategies (blue-green, canary, rolling) for shipping changes without downtime, and handling secrets safely inside a CI environment.

1. What CI/CD Actually Automates

Continuous Integration means every code change is automatically built and tested as soon as it's pushed, catching integration problems (a change that breaks something elsewhere in the codebase) within minutes rather than being discovered days later when several people's changes collide. Continuous Deployment (or the more conservative Continuous Delivery, which stops just short of automatic production deploys, requiring a manual approval step) extends this by automatically deploying code that passes all checks, removing manual, error-prone deploy steps and making shipping small changes frequently the normal, low-risk default instead of a rare, high-stakes event.

2. Anatomy of a Pipeline

A typical pipeline runs as an ordered sequence of stages, each of which must succeed for the pipeline to proceed: install dependencies, run linting/static analysis, run the automated test suite, build the production artifact (a compiled bundle, a Docker image), and finally deploy that artifact to an environment. Structuring it as distinct, ordered stages means a failure is immediately localized to a specific step (a lint failure won't waste time running a slow test suite afterward if the pipeline is ordered to fail fast on cheaper checks first).

3. GitHub Actions Workflow Basics

A GitHub Actions workflow is a YAML file in `.github/workflows/` that defines when it runs (on: push, pull_request, a schedule, etc.), one or more jobs, and, within each job, a sequence of steps — either a shell command (run:) or a reusable community action (uses:, like actions/checkout to pull the repo's code onto the runner). Jobs can run in parallel by default, or be made to depend on each other via needs:, letting you express, for example, that a deploy job should only run after a test job has succeeded.

YAML

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: "npm"
      - run: npm ci
      - run: npm run lint
      - run: npm test
```

4. Tests and Linting as Real Gates

A pipeline stage is only a meaningful gate if a failure actually blocks the pipeline from proceeding — branch protection rules on the repository that require the CI check to pass before a pull request can be merged are what turn 'we have tests' into 'broken code literally cannot reach main.' A test suite that exists but that failures are routinely ignored or overridden on is functionally decorative; the discipline of never merging on red CI is what makes the whole practice worth the setup investment.

5. Caching Dependencies for Faster Builds

Reinstalling every dependency from scratch on every single pipeline run wastes real time and, for many package registries, real bandwidth — most CI platforms support caching a dependency directory (node_modules, or more precisely the package manager's download cache) keyed by a hash of the lockfile, so the cache is reused automatically whenever the lockfile hasn't changed and invalidated automatically the moment it has. This single optimization is often the difference between a pipeline that takes 30 seconds versus several minutes on every single push.

6. Deployment Strategies

A rolling deployment gradually replaces old instances with new ones a few at a time, keeping the service available throughout but meaning both versions briefly serve traffic simultaneously. A blue-green deployment runs the new version fully in parallel on a separate environment, then switches all traffic over at once (typically via a load balancer or router change) once it's verified healthy — enabling an instant rollback by simply switching back. A canary deployment routes a small percentage of real traffic to the new version first, monitoring for errors before gradually increasing that percentage — catching a bad deploy while it's only affecting a small fraction of users instead of everyone.

7. Secrets in CI Pipelines

A CI pipeline frequently needs secrets (a deploy key, a database password, an API token) to do its job, and these must never be hardcoded into the workflow file itself, since workflow files are typically committed to the same repository they're building. CI platforms provide a dedicated, encrypted secrets store (GitHub Actions Secrets, for instance) that injects secret values as environment variables at runtime without ever exposing them in the workflow file's source or, by default, in the pipeline's logs — and it's worth verifying that a secret value is never accidentally echoed or logged by a script step, since that would defeat the protection entirely.

Common Pitfalls

- Having a test suite in the pipeline that isn't actually enforced by branch protection, so failing tests don't stop broken code from merging.
- Deploying directly to production with no staging environment or canary/rolling rollout, so a bad deploy immediately affects every user at once.
- Not caching dependencies between pipeline runs, wasting minutes reinstalling the exact same packages on every single push.
- Hardcoding a secret directly into a workflow YAML file instead of using the CI platform's encrypted secrets store.
- Only running the pipeline on merges to main, missing the fast feedback a pull-request-triggered run would give before code is even merged.
- Tolerating flaky tests by re-running them until they pass instead of fixing the underlying flakiness, eroding trust in what a failing test actually means.
- Having no rollback plan for a failed deployment, turning a bad deploy into an extended outage while a fix is rushed out instead of simply reverting.

Key Takeaways

- CI catches integration problems within minutes of a push; CD automates getting passing code safely into production.
- Structure pipelines as ordered stages that fail fast on cheap checks (lint) before running expensive ones (full test suite, build).
- A test gate is only real if branch protection actually blocks merging on a failing check — otherwise it's decorative.
- Dependency caching keyed on the lockfile hash is usually the single biggest lever for pipeline speed.
- Blue-green and canary deployments both enable safer rollouts and fast rollback compared to an all-at-once deploy.
- Secrets belong in the CI platform's encrypted secrets store, never hardcoded in a workflow file or echoed into logs.

Learn More

GitHub Docs — Understanding GitHub Actions

<https://docs.github.com/en/actions/get-started/understand-github-actions>

GitHub Docs — Using Secrets in GitHub Actions

<https://docs.github.com/en/actions/security-guides/using-secrets-in-github-actions>

Martin Fowler — Continuous Integration

<https://martinfowler.com/articles/continuousIntegration.html>